

JIO Micro Monitoring Log

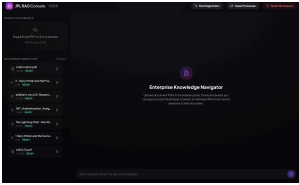
This log is intended for high-granularity tracking of daily internship activities. Consistent updates ensure transparency and provide a historical record of technical growth and operational contributions.

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jun 1, 2026	Before Lunch	- Explored the RAG optimisation techniques - Read HNSW, INF-PQ and SCANN along with the trade offs - Devised an approach suitable for my use case	Understanding the technicalities of these algorithms and their potential bottle-necks Took help of documentation and blogs to get an abstract of everything	Type detailed text here
	After lunch Session 1	- Completed the local Milvus configuration - Updated the RAG architecture - Implemented and verified the /chat functionality with simple chunking and brute approach		
	After lunch Session 2	- Google-generativeai integrated		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		- Few short prompting implemented		
	Home Session	- Delete One Document endpoint - Delete All Documents endpoint - Optimized Embedding Loading	Fixing Milvus Search Failure needed AI care. I was accidentally Regenerated Embeddings On every Startup which I fixed	
Jun 2, 2026	Morning Session (Before lunch)	- Migrated to postman for testing  - The Alchemist book pdf is tested for a discreet query - Set up future issues to resolve in github in advanced version milestone of the project - INITIAL SCOPE OF PROJECT IS IMPLEMENTED		
	After Lunch (+home)	- Few errors arise - Migrated to standalone milvus db and attu via docker	Encountered the issue of recurring python daemon threads which made the	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		- Made delete /documents asynchronous - Improved the reset functionality. Tried to make sure that the indexing starts from 1 after every reset	startup of the server to halt indefinitely. I remotely deleted the branch for resolving issue via github, but forgot to change branch in VS code. Kept on doing changes on that deleted branch. resolved it by introducing another branch on the deleted one and then merging it with main.	
Jun 3, 2026	Before lunch	- Debugging and verification of endpoints - Postgres Validated - Tried optimising the /documents endpoint but it failed	System was hanging a lot which led to a full blown debugging of all the apis. Multiple processes were created in middle which held resources. This was because of websockets (which were of no use in the current model) so got rid of them	
	After Lunch	- Tried solving the DB issue - The issue: after hitting /upload the file is chunked up and embeddings are made and stored in milvus. However when inspected through Attu, the collections and rows		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		are loaded but NO DATA IS VISIBLE (Thiis issue is still not solve) - Started brushing-up docker		
Jun 4, 2026	All Day	• Backend Setup: Moved all backend code into a dedicated Project/backend folder. • Frontend Development: Set up a new React frontend using Vite in the Project/frontend folder. • Backend Improvements: Added CORS support for local development, fixed a text encoding error that caused crashes, and added safety checks to stop the system from hanging during searches. • System Maintenance: Added a new clean-system bat command to clear memory and stop background processes that get stuck. • User Interface: Built a new dark-mode interface with features like real-time document status updates, citation previews, and control buttons for diagnostics and system resets. • Developer Tools: Added a new script to test APIs from the command line, created a Windows batch file to stop		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		stuck processes, and wrote documentation explaining the backend flow. 		
Jun 5, 2026	Before Lunch	- Validated the previous flow and shown ma'am the categorical optimization	- Learning is that how to simplify complex workflows into understandable concepts	
	After Lunch Session1	- Relational Database Migration: Added category column to Postgres Document model and performed non-destructive database migration. - Implemented categorical_chunks in Milvus, added category search functions, and added support for filtering vector matches by document ID. - Automated Document Ingestion Pipeline: Updated upload endpoint for category support, integrated Gemini for automatic classification, and implemented		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		automated summary embedding tasks.		
	After Lunch Session2	- Two-Stage Chat Query Routing: Implemented a router in llm_service.py for category-specific retrieval and added a confidence triage threshold for global fallback. - User Interface Upgrades: Added category badges, manual category input, and interactive Category Workspace filter buttons in the sidebar. - Error Tolerance & Troubleshooting: Added traceback logging to fallback to handle LLM rate limits - Testing Remains.		
Jun 6, 2026	Self-study	- Read up on multi-tenant DB design patterns, knew Groups was coming up next - Just background reading, nothing built yet	Just background reading, nothing built yet	
Jun 7, 2026	Self-study	- Sketched out the Group/GroupMember schema on paper before touching code	Wanted a clear plan before rushing into the groups layer	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		- Wanted a clear plan before rushing into the groups layer		
Jun 8, 2026	Before lunch	- Renamed Project/ → backend/, decided this is now the "untouchable" core engine going forward - Rewrote backend_flow.md to keep the pipeline documented properly - Merged the categorical-routing branch into main — categorical RAG officially live	Realised my docs were already getting stale, had to re-trace the whole flow myself	
	After lunch	Merged the categorical-routing branch into main — categorical RAG officially live	Small merge conflicts in main.py, nothing major	
Jun 9, 2026	Session	- Read up on JWT best practices, weighed bcrypt vs argon2, stuck with bcrypt for now - Auth was new territory for me, wanted to understand it before touching the real repo	Auth was new territory for me, wanted to understand it before touching the real repo	
Jun 10, 2026	Session	- Tinkered with a basic auth flow outside the main	Not reflected in git, just me getting hands dirty first	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		repo, just to get comfortable before wiring it in - Not reflected in git, just me getting hands dirty first		
Jun 11, 2026	Session	- Played around with FastAPI's OAuth2PasswordBearer, took a while to understand how it talks to postman - Bit of a learning curve, mostly trial and error	Bit of a learning curve, mostly trial and error	
Jun 12, 2026	Before lunch	- Migrated frontend from JSX to TSX, more painful than expected - Started scaffolding live/backend/ as a separate FastAPI app - Built out auth.py, config.py, database.py, schemas.py for the new Live layer - Got register → login → JWT working end-to-end, tested by pasting the token into jwt.io - Merged M0 monorepo setup to main (PR #37)	Had to decide how live/backend/ would reuse the core engine without copy-pasting code — went with adding backend/ to sys.path	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
	After lunch	Built out auth.py, config.py, database.py, schemas.py for the new Live layer; Got register → login → JWT working end-to-end, tested by pasting the token into jwt.io	Small wins like this feel good when you're solo on a project this size	
	Evening	Merged M0 monorepo setup to main (PR #37)	M0 and M1 (Auth) both officially done today	
Jun 13, 2026	Session	- Planned out the Groups schema and how group-scoping should cascade across Postgres + Milvus + disk - Wanted a clean plan before building, given how much touches this layer	Wanted a clean plan before building, given how much touches this layer	
Jun 14, 2026	Session	- Tried a couple of approaches for the invite flow before settling on a simple ordered validation chain - Went back and forth a bit before landing on something clean	Went back and forth a bit before landing on something clean	
Jun 15, 2026	Session	- Cleaned up leftover files from earlier testing in the uploads folder that were confusing my queries	Nothing commit-worthy, just groundwork so Groups wouldn't be built on a messy base	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		- Nothing commit-worthy, just groundwork so Groups wouldn't be built on a messy base		
Jun 16, 2026	Session 1	- Built out Groups: models, CRUD, invite-by-email, group-scoped document routes, group-scoped /chat - Tested that querying one group never leaks another group's content - Added the categories endpoint for the frontend dropdown - Built 3-mode chat: pin a document, pin a category, or let it auto-route - Wrote proper tests this time — 5/5 passing, merged M2 + 3-mode routing to main (PR #38)	Spent extra time double checking the leakage case, didn't want to find out the hard way later	
	Session 2	Added the categories endpoint for the frontend dropdown; Built 3-mode chat: pin a document, pin a category, or let it auto-route; Wrote proper tests this time — 5/5 passing	Merged M2 + 3-mode routing to main today	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jun 17, 2026	Before lunch	- Realised the one-category-per-document design was too limiting (e.g. Harry Potter 1 & 2 had zero relation to each other) - Started sketching a many-to-many category model on paper - Removed the old category column off Document, started building the Category and DocumentCategory tables	Had to rethink something I'd already built, a bit annoying but better to fix now than later	
	After lunch	Removed the old category column off Document, started building the Category and DocumentCategory tables		
Jun 18, 2026	Session	- Built <code>_extract_summary_text_from_pdf()</code> — pulling first 5 + last 2 pages for better classification context - First pass at <code>consolidate_categories()</code> so Gemini can group related categories together (e.g. "Novels", "Harry Potter Books")	Took a few tries to get the prompting right so Gemini wouldn't over-group or under-group things	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jun 19, 2026	Session	- Added group_id as a scalar field on the categorical_chunks collection in Milvus so categories can't leak across groups - Wrote cleanup logic so a Category row only gets deleted once zero documents remain in it	Almost shipped a bug where deleting one doc from a shared category would wipe the whole category	
Jun 20, 2026	Session	- Ran the Postgres migration for the new category tables - Wrote an end-to-end integration test for the whole multi-category flow - Renamed backend/ to core_backend/ on disk and pointed the live adapter's sys.path to it - Left a basic WebSocket echo stub in place before heading off	Work is solid and tested but I haven't committed it yet — wanted to wrap up clean before my holiday	
Jun 30, 2026	Morning	First day back from leave, caught up on where I left off Committed all the uncommitted work that had piled up before the break — the	Had to carefully stage the rename separately since git was treating it as delete + untracked, didn't want to mess it up	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		core_backend/ rename, multi-category schema changes, live backend path fixes		
	After lunch	Set up the categorical_chunks collection in Milvus properly for storing category summary embeddings Wired up the two-stage query router — queries category summaries first to narrow down the retrieval space, then does the actual chunk search	The two-stage idea makes a lot of sense in hindsight, surprised I didn't design it this way from the start	
Jul 1, 2026	Before lunch	Added Group DB models and member schemas to the live backend Built _assert_membership() as a strict guard — every single group endpoint now runs through this before doing anything	Wanted this centralised rather than copy-pasting the membership check into every route	
	After lunch	Wired up the multi-category grouping logic and document association in the live backend	A lot of pieces came together today, felt like the system finally started looking like what I had designed on paper	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		Connected the 3-mode chat routing (doc pin / category pin / auto-routing) to the new multi-category schema		
Jul 2, 2026	Morning	Started building out the WebSocket layer properly — ws.py went from echo stub to an actual persistent server Implemented token-by-token answer streaming so the LLM response types out in real time rather than arriving all at once	Understanding the difference between HTTP and WebSocket properly took a bit — HTTP is request-response, WS is a persistent TCP channel where either side can push anytime	
	After lunch	Built the dynamic category consolidation algorithm — after each ingestion, Gemini reviews all categories in the group and proposes parent groupings, output parsed from JSON via regex Added a solid rate-limit fallback for category summary generation so a Gemini 429 doesn't break the whole ingestion pipeline	Free tier Gemini (5 req/min) gets hit almost immediately when categorisation + consolidation + summary all fire together — fallback was essential	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jul 3, 2026	Evening	Bumped top_k ceiling from 10 to 100 for global search Added bypass_llm toggle and MOCK_LLM env flag to skip Gemini calls entirely during testing	Realised I was burning through API quota just verifying retrieval logic — needed a way to test without touching Gemini at all	
	Morning	Fixed a critical background task bug — was using asyncio.to_thread on an async def function which meant the coroutine was never actually awaited, ingestion pipeline was silently failing	Took a while to spot this one. asyncio.to_thread is for blocking sync functions only, not for coroutines. Once I understood that, the fix was one line	
	After lunch	Removed the redundant group_id from the Pydantic ChatRequest body — it's already in the URL path, having it in the body too was causing 422 errors when clients didn't repeat it Enforced uniqueness on Group.name at the DB level and wired a clean 400 response for duplicate group names	Lesson learned: URL path variables are for resource scoping, request body is for payload only — mixing them up causes quiet validation failures	

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
	Evening	Expanded bypass_llm across the full stack category routing LLM, generation LLM, all skipped when toggled. Now have a true 100% token-free debug mode Cleaned up the repo with a proper .gitignore to remove all the clutter from untracked files that had been building up	Should have done the .gitignore cleanup much earlier honestly	
Jul 4, 2026	Self-study	Read about multi-tenancy patterns and how large SaaS platforms handle group-level data isolation at scale. The core question I was sitting with: how do you bolt a tenancy layer onto a system that was never designed for it, without corrupting the original system's integrity? The answer I kept coming back to was the adapter pattern. Keep the core engine oblivious to groups entirely and intercept at the boundary.		
Jul 5, 2026	Self-study	Sketched out the Live Backend Adapter architecture. The mental model I settled on: the core engine is a library, not a service.		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		It should know nothing about users, sessions, or groups. The Live layer owns all of that and simply delegates retrieval work downward. Getting this boundary right on paper before touching code felt important. Most complexity in systems comes from unclear ownership boundaries.		
Jul 6, 2026	Session	Started scaffolding the Live adapter endpoints for registration, login and JWT issuance. Made the conscious architectural call that group logic lives entirely at the adapter level. The core engine processes documents, it does not care who owns them. Keeping these concerns separated means the core engine stays independently testable and reusable. Mixing them in would have created a coupled mess that's very hard to unpick later.		
Jul 7, 2026	Session	Implemented groups.py and documents.py in the Live layer. Every document now carries a group_id from the moment it enters the system. Had to		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		re-wire process_document_task_with_ws to pass group context through while still delegating the actual processing to the core engine cleanly. The tricky part was making the bridge feel invisible. The core engine should receive a task and process it without any awareness that a group constraint exists above it. Got that working.		
Jul 8, 2026	Session	Built group membership verification as middleware enforced at the DB level, not just in application logic. There is a meaningful difference between the two: application-level checks can be bypassed by hitting a different route, DB-level constraints cannot. Added an endpoint to inspect user-group mappings in one call during debugging. Debugging multi-tenant systems is painful without observability. The mapping endpoint was about making the tenant state visible so I am not flying blind during ingestion tests.		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jul 9, 2026	Morning	Built a proper HTML test console with a split-screen layout for core and live backends independently. Added Mermaid flowchart rendering so the architecture stays visible while testing. Having the system diagram live next to the API buttons changes how you think about what you are testing. Postman works but it has no spatial relationship to the system. A custom console lets you test with the mental model literally on screen.		
Jul 9, 2026	After lunch	Added Bypass LLM toggles to both consoles and a DB reset button on the live side. Built an endpoint that surfaces all categories, their LLM-generated summaries, and their associated documents in a single call. This became the primary verification step after every ingestion run. The Bypass LLM toggle is philosophically important. It decouples retrieval quality from generation quality so both can be evaluated independently without burning API quota.		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
Jul 10, 2026	Morning	Hit a fundamental limitation of pure dense vector search: it is excellent at semantic similarity but completely blind to exact keyword matches. A user querying a specific name or ID gets back semantically adjacent results, not the right ones. Integrated rank-bm25 as an in-memory sparse indexer to solve the lexical gap. Chunks are tokenized lazily at startup. pip dependency resolution failed inside the venv when installing rank-bm25. Fixed by targeting the environment's Python executable directly rather than relying on system pip. Small but the kind of thing that wastes an hour if you don't know where to look.		
Jul 10, 2026	After lunch	Implemented Reciprocal Rank Fusion to merge BM25 and Milvus results into a unified ranking. Then added cross-encoder/ms-marco-MiniLM-L-6-v2 as a second-stage reranker. The key insight is that the first stage (BM25 + dense) is about recall, casting a wide net.		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		The cross-encoder is about precision, looking at query and chunk together and scoring their actual interaction. Cross-encoder inference on large windows is expensive. Constrained it to the top 15 to 45 merged candidates so the latency stays sub-second. Full pipeline now: BM25 union Milvus, RRF merge, cross-encoder rescore.		
Jul 10, 2026	Evening	Rewrote root README and added FLOWS.md with sequence diagrams covering all major user scenarios. Merged both PRs. The retrieval stack now has three distinct layers each doing something the others cannot.		
Jul 11, 2026	Self-study	Read up on HNSW graph indexing properly. I had skimmed it on Jun 1 but it was abstract then. Reading it now with the actual retrieval pipeline in my head was completely different. The skip-graph structure finally clicked as a geometric intuition, not just an algorithmic one. The same		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		concept reads differently depending on what problem you are trying to solve with it. Context matters more than the material itself.		
Jul 12, 2026	Self-study	Went deep on Milvus HNSW configuration parameters. M controls the number of edges per node in the graph and efConstruction controls search width during the index build. Higher values give better recall at the cost of memory and build time. Planned the values and wrote a note to self that HNSW will not show measurable latency gains at roughly 2500 chunks. Worth doing architecturally but benchmarks will mislead at this scale. The danger with premature optimization is not just wasted time but false confidence. Documenting the scale caveat explicitly felt important.		
Jul 13, 2026	Morning	Upgraded the Milvus index from FLAT to HNSW and configured parameters via config files. FLAT is O(N) brute force, every query scans every vector. HNSW is		

Date	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Manager Remarks/Feedback
		O(log N) skip-graph navigation, queries traverse a hierarchical graph structure to find approximate nearest neighbours. The upgrade is architectural even if the current dataset is too small to feel the difference. Documented the scale expectations explicitly in README. The right time to put proper infrastructure in place is before you need it, not after. At 100k chunks this would matter enormously and retrofitting an index type in Milvus is not trivial.		
Jul 13, 2026	After lunch	Built a confidence gate in the routing logic. If the top cross-encoder score falls below the configured threshold the system skips the Gemini call entirely and returns a graceful no-answer response rather than generating something plausible but wrong. Hallucination is a retrieval problem masquerading as a generation problem. If the right context is not retrieved confidently, the LLM should not be trusted to fill the gap. Added LOG_RETRIEVAL_SCORES flag to		

	Time Segment	Task/Activity (What & How)	Challenges & Resolution	Ma Ren
		print score distributions in the terminal so thresholds can be tuned empirically rather than guessed. Hardcoding a threshold without observability is just guessing with extra steps. The logging flag turns it into an iterative calibration process.		

